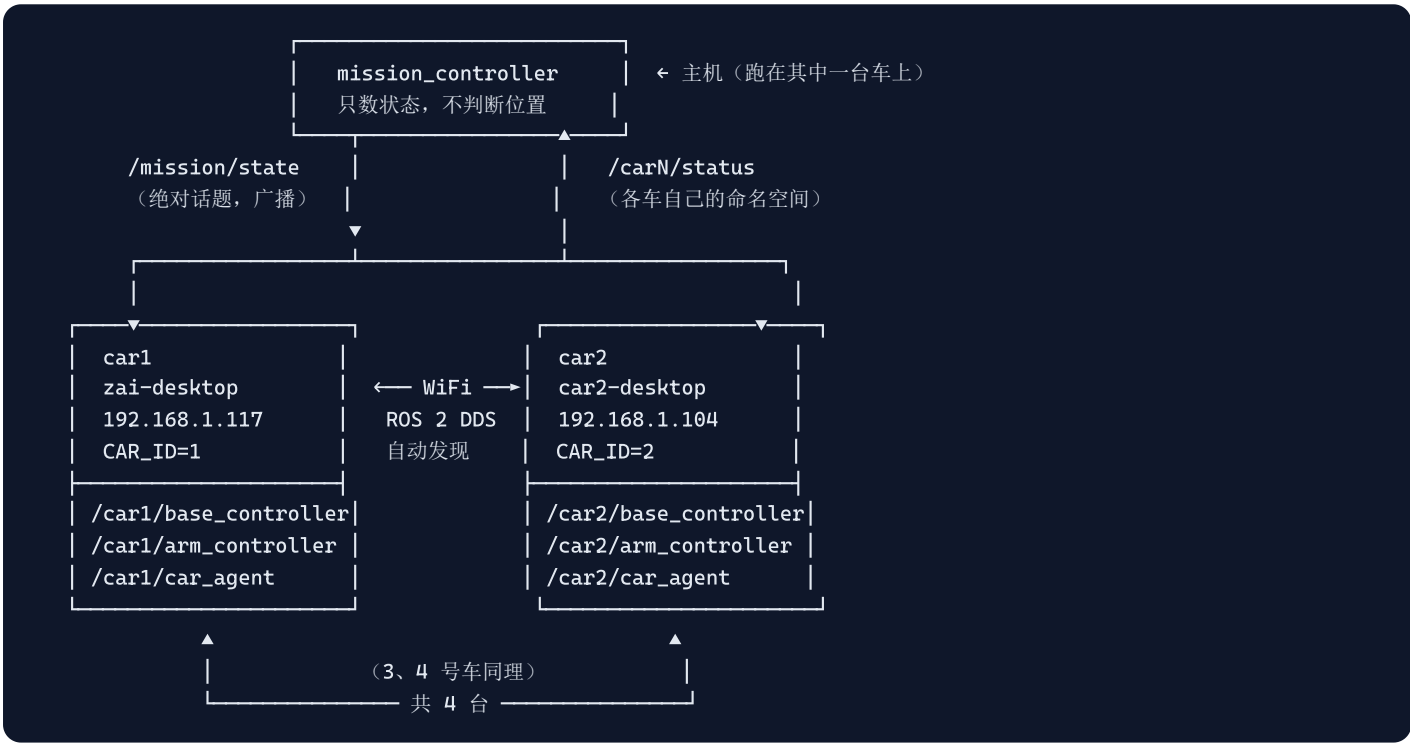
四车协同 · 卷尺桥 · 完整逻辑链

> 从「你敲一条命令」到「舵机/电机真的动了」，中间每一跳都拆开。 分三层看：**单台车内部 → 车与主机之间 → 车与硬件之间。**
>
> 文中所有拓扑都是**两台真机跑起来实测抓的**，不是推导。

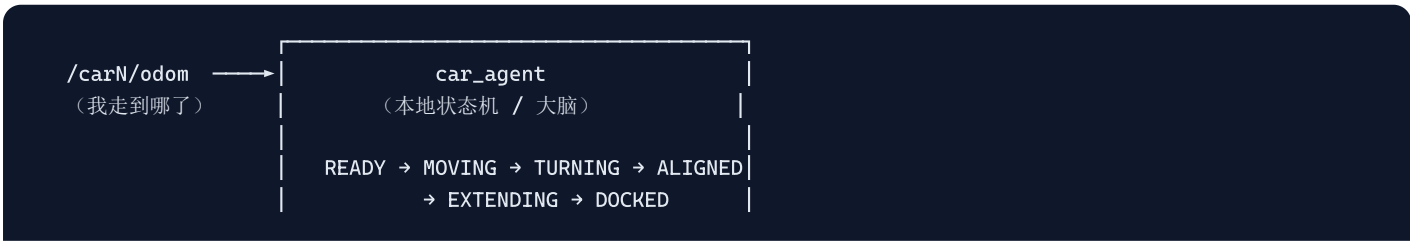
一、整体拓扑

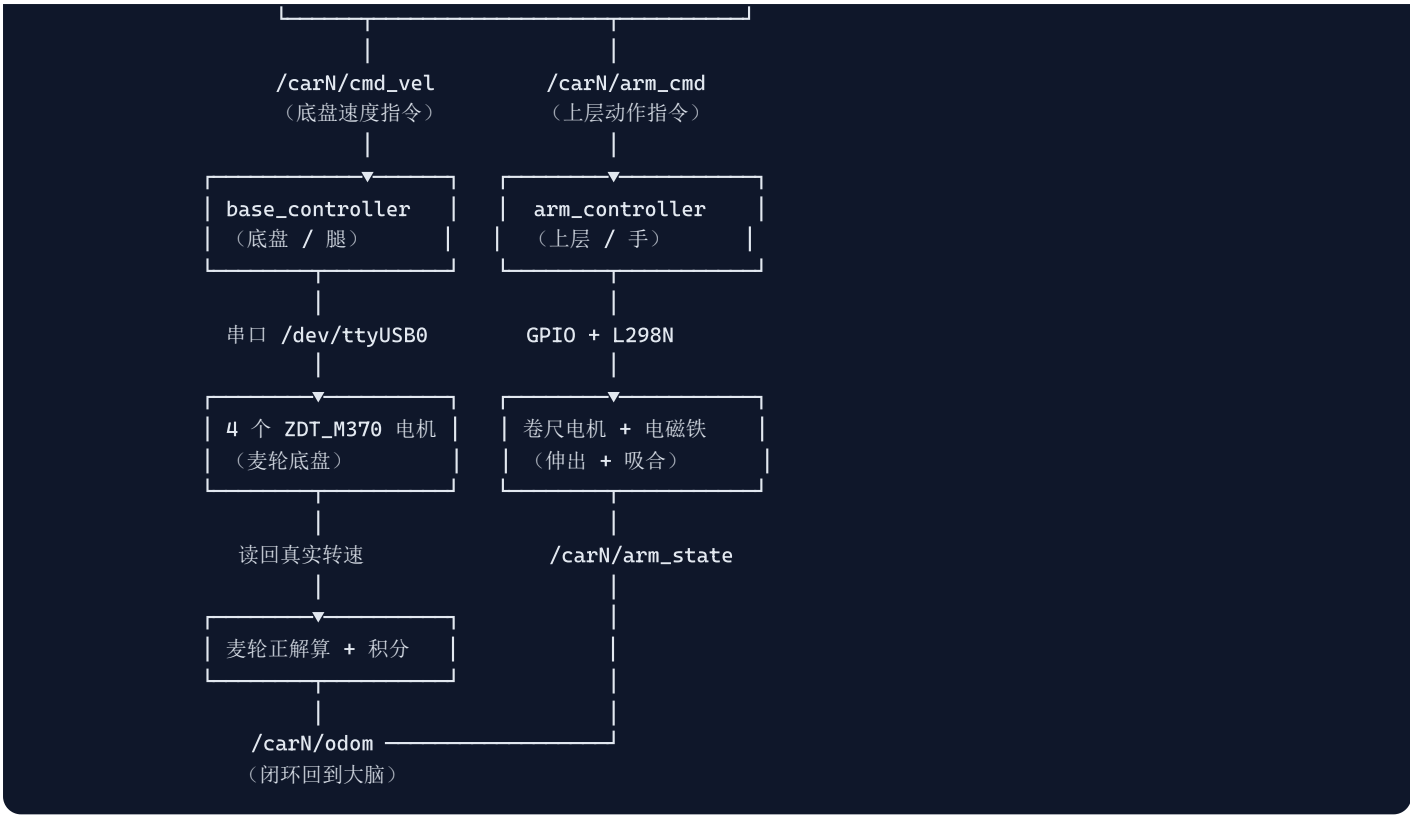


关键设计：4 台车跑**同一份代码**，区别只有 `CAR_ID` 环境变量。所以不存在"副车代码"。

二、单台车内部逻辑链

每台车跑 **3 个节点**，一进两出：





2.1 三个节点的职责

节点	一句话	订阅	发布
`car_agent`	**大脑**：决定"什么时候做什么"	`/carN/odom`、`/mission/state`	`/carN/cmd_vel`、`/carN/arm_cmd`、`/carN/status`
`base_controller`	**腿**：把速度指令变成四个轮子转	`/carN/cmd_vel`、`/carN/imu`	`/carN/odom`

节点	一句话	订阅	发布
`arm_controller`	**手**：伸出卷尺、吸合电磁铁	`/carN/arm_cmd`	`/carN/arm_state`

2.2 车内的两个闭环

闭环 ①：底盘速度环（30 Hz）

```
car_agent 发 /carN/cmd_vel（前进 0.25 m/s）
      ↓
base_controller 麦轮逆解算 → 四个轮子各转多快
      ↓
按地址发 4 条 ZDT 速度指令（F6 帧）
      ↓
电机内部 PID 闭环（**电机自己保证转速**，不用我们写）
```

闭环 ②：里程计反馈环（20 Hz）

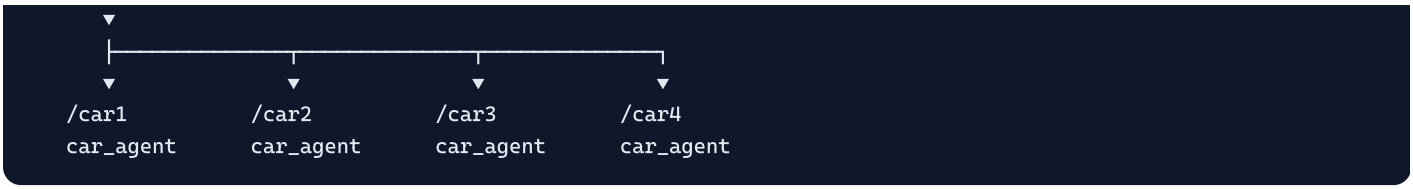
```
base_controller 逐个问 4 个电机"你现在转多快"
      ↓
电机回真实转速（内部编码器测的，不是我们发的目标值）
      ↓
麦轮正解算 → 车实际速度 → 按时间积分
      ↓
发 /carN/odom
      ↓
car_agent 判断「走够 3 米没」← 回到起点
```

这就是为什么读数要读回来而不是用目标值：打滑了、爬坡了，真实转速会变，里程计才准。

三、对外通信闭环

3.1 下行：主机 → 车（一条广播打全部）

```
mission_controller
      |
      |  /mission/state    ← ★ 绝对话题名，不受命名空间影响
      |  data: "GO" / "EXTEND" / "ABORT"
```



一条指令，四台车收到，各自按自己的 `CAR_ID` 决定干什么：

收到的	car1 / car2 做	car3 / car4 做
`GO`	先直行 3 米 → 再转向	**直接转向**（不直行）
`EXTEND`	伸卷尺 → 电磁铁吸合	同左
`ABORT`	进 ERROR 状态	同左

3.2 上行：车 → 主机（各自上报）



每台车都往**自己命名空间下**的 `status` 发，所以主机能分清是谁。

3.3 闭环是怎么形成的



关键点：主机不问"你走到哪了"，只数"你上报了吗"。 所以：

- 不需要时钟同步（各车开机时间不同无所谓）
- 加第 5 台车不用改主机逻辑

- 任何一台车卡住 → 30 秒超时 → 报出是哪台车、卡在什么状态

3.4 两条必须记住的通信规则

① 状态上报要周期性重发，不能只发一次

WiFi 丢一个包，主机就永远等不齐 → 整个任务卡死。所以 `car_agent` 每秒重发一次 `status`。

② 主机也要周期重发指令

发 `GO` 的那一包如果丢给某台车，它就一直停在 READY，主机等不到它。所以 `mission_controller` 每秒重发当前阶段的指令。

四、完整任务时序

```
时刻      事件                                          涉及
──────────────────────────────────────────────────────────────────
t=0.0     主机启动，autostart_delay=4s 倒计时          mission_controller
t=1.0     各车启动，报 READY                           4 × car_agent
t=4.0     主机集齐 4 个 READY → 广播 GO                mission_controller
          │
t=4.1     ├─ car1/car2 收到 GO → MOVING（先直行）      car1/2 agent
          └─ car3/car4 收到 GO → TURNING（直接转）      car3/4 agent
          │
          每 0.05s：发 cmd_vel 给底盘                   car_agent → base_controller
          每 0.05s：底盘读回转速、算里程计、发 odom      base_controller
          │
t=16.1    car1/car2 里程 ≥ 3.0 米 → MOVING→TURNING     car1/2 agent
t=17.5    car3/car4 航向到位 → TURNING→ALIGNED         car3/4 agent
t=19.2    car1/car2 航向到位 → TURNING→ALIGNED         car1/2 agent
          │  各自上报 /carN/status = ALIGNED
          │
t=19.3    主机集齐 4 个 ALIGNED → 广播 EXTEND          mission_controller
          │
t=19.4    4 台车 → EXTENDING                           4 × car_agent
          发 arm_cmd=EXTEND → arm_controller
          arm_controller：L298N 给卷尺电机通电 8 秒
          │
t=27.4    伸出时间到 → 停电机 → 电磁铁通电 → DOCKING
t=28.4    等 1 秒吸稳 → DOCKED → 上报
          │
t=28.5    主机集齐 4 个 DOCKED → 广播完成              mission_controller
          │
          ▼
      === DONE ===
```

> **注意**：以上时间是基于仿真的估算。真机上 `MOVING` 那 12 秒取决于直行距离和速度，`EXTENDING` 那 8 秒是你实测出来的 `arm.extend_seconds`。

五、硬件层逻辑链

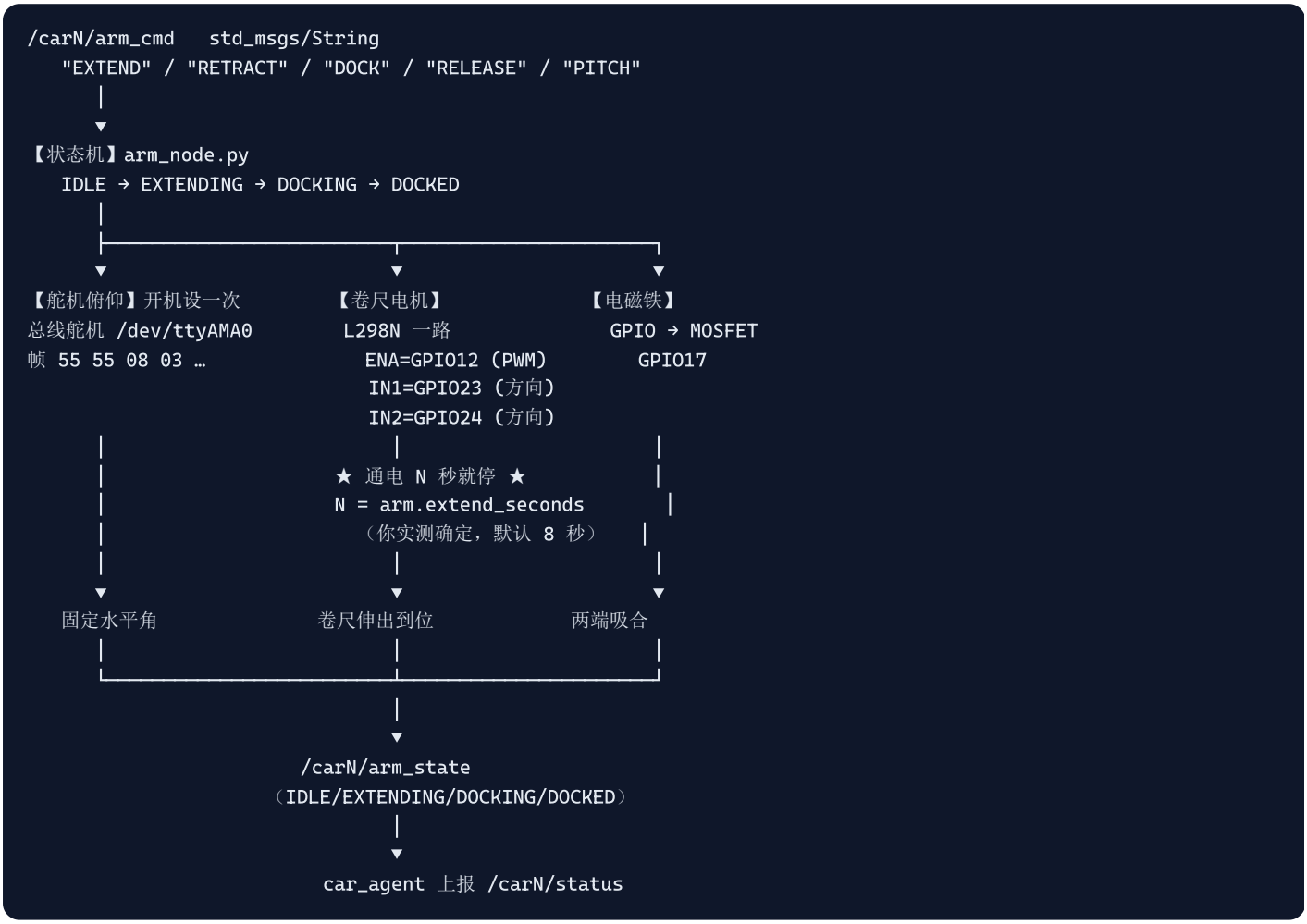
5.1 底盘（4 个 ZDT_M370 闭环伺服 + 麦轮）

```
/carN/cmd_vel   geometry_msgs/Twist
   linear.x (m/s)  前进
   linear.y (m/s)  横移
   angular.z (rad/s) 自转
      |
      ▼
【麦轮逆解算】mecanum.py
   w_FL = (vx − vy − wz·L) / r        r = 0.0485 m（轮径 97mm）
   w_FR = (vx + vy + wz·L) / r        L = Lx + Ly = 0.08006 + 0.1385
   w_RL = (vx + vy − wz·L) / r              = 0.21856 m
   w_RR = (vx − vy + wz·L) / r
      |
      ▼
【限幅】等比例缩放（保方向，不是截断）
      上限 16 rad/s ≈ 车速 0.81 m/s
      |
      ▼
【单位换算】1:1（驱动器已按减速比缩放）
      rad/s → RPM = w × 60 / (2π)
      |
      ▼
【拼 ZDT 速度帧】zdt_driver.py
      01 F6 方向 加速度(2B) 速度(2B) 同步 6B
      |   ↑ 地址 1/2/3/4 对应 FL/FR/RL/RR
      ▼
串口 /dev/ttyUSB0 @115200
      |
      ▼
扩展板 → 4 个电机同时转
      |
      ├═══ 电机内部 PID 闭环（我们不用管）═══╗
      |                                      ‖
      ▼                                      ‖
【读回真实转速】读命令 Addr 35 6B            ‖
      回：Addr 35 符号 转速(2B,0.1RPM) 6B    ‖
      |                                      ‖
      ▼                                      ‖
【麦轮正解算】                               ‖
   vx = r/4 · ( w_FL + w_FR + w_RL + w_RR)   ‖
   vy = r/4 · (−w_FL + w_FR + w_RL − w_RR)   ‖
   wz = r/(4L) · (−w_FL + w_FR − w_RL + w_RR)‖
      |                                      ‖
      ▼                                      ‖
【里程计积分】odometry.py                    ‖
   机体系 → 世界系（用当前朝向旋转）          ‖
   朝向：有 IMU 信 IMU，没有就用轮速积分      ‖
      |                                      ‖
      ▼                                      ‖
   /carN/odom  ──────────────────────────────╝
```

⚠️ **三个必须记住的点**

点	说明
减速比要写进驱动器	手册 5.5.7：设了 18.8 后收发都是**输出轴**的量，树莓派不用再乘除。不设会差 18.8 倍
只有地址 1 回确认	运动命令只有地址 1 会应答。对 2/3/4 等应答会白等 timeout，控制环直接废掉
读回的是真实值	电机闭环报的真实转速，不是我们发的目标值 —— 所以里程计比开环估算准

5.2 上层（JGY-370 卷尺电机 + 电磁铁）



⚠️ 三个必须记住的点

点	说明
两个串口别搞混	底盘 `/dev/ttyUSB0`（USB转串口），舵机 `/dev/ttyAMA0`（GPIO14/15，物理脚 8/10）
L298N 的 ENA 跳线帽要拔掉	插着的话 ENA 常高，**PWM 静默失效**（电机只会全速），不报错
GPIO 不能直拉电磁铁	线圈电流安培级，必须经 MOSFET。**续流二极管和 10k 栅极下拉必须有**，MOSFET 要选逻辑电平的

六、当前实际部署状态

```
✅ car1  zai-desktop   192.168.1.117  用户 zai    CAR_ID=1  ROS Jazzy(desktop)
✅ car2  car2-desktop  192.168.1.104  用户 car2   CAR_ID=2  ROS Jazzy(ros-base)
⏳ car3  （未部署）
⏳ car4  （未部署）
```

已验证的（真机实测）

项	状态
单机自检 `test_no_hardware.sh`	两台都是 **15/15**
跨机双向通信	✅ `car1→car2` 和 `car2→car1` 都通
两车完整任务流程	✅ `GO → 各自移动/转向 → ALIGNED → EXTEND → DOCKED → DONE`
话题命名空间隔离	✅ `/car1/odom` 和 `/car2/odom` 各自 1 个发布者，互不干扰
故障注入（杀一台车）	✅ 30 秒超时 → 报出是哪台车 → 广播 ABORT

还没验证的

项	为什么
硬件标定	电机没到货（麦轮方向、地址、减速比、卷尺时间、磁铁极性）
IMU	驱动还没写（`base_node` 订阅 `/carN/imu` 但没人发布）
4 台车同时跑	只有 2 台
急停	还没实现

七、踩过的坑（跑起来才发现的）

#	现象	真因
1	`Failed to normalize ... '<class 'int'>'`	launch 的 `parameters` 里塞了裸 int/bool/float，要包 `ParameterValue`
2	`value='1' is not an instance of <class 'int'>`	`ParameterValue` 是**校验**不是**转换**，静态值传原始对象
3	`not a list of one of the allowed types '[[1,2],[3,4]]'`	**ROS 2 参数不支持嵌套数组**
4	`not one of the allowed types '{}'`	**ROS 2 参数不支持字典**
5	`AttributeError: ... '_TYPE_SUPPORT'`	**名字遮蔽**：自己的类 `Odometry` 撞了 `nav_msgs/Odometry`

#	现象	真因
6	`NameError: ALIGNED is not defined`	「编排阶段常量」和「车辆上报状态」是两套东西，混用了
7	后两台车也去直行了	参数写在配置里，**代码里压根没读**
8	`serial-getty` 占串口 → 权限 `root:tty` 无读	Ubuntu 把串口当登录控制台
9	**两台车话题全在根命名空间，`/odom` 混在一起**	`car.launch.py` **漏了** `namespace=` —— 单车测试发现不了，两台真机一起跑才暴露

第 9 条是今天抓到的，也是最典型的"只有规模上去才会暴露"的 bug。

八、一句话总结整条链

```
你敲命令
  → 主机广播 /mission/state
  → 4 台车各自的 car_agent 收到，按 CAR_ID 决定做什么
  → 发 /carN/cmd_vel 给 base_controller
  → 麦轮逆解算 → ZDT 速度帧 → 串口 → 电机转（电机内部闭环）
  → 编码器读回 → 麦轮正解算 → 里程计积分 → /carN/odom
  → car_agent 看 odom 判断"到位没" → 上报 /carN/status
  → 主机集齐 4 个 → 广播下一步
  → 循环，直到 DONE
```

全程没有一处依赖"过了几秒"，全部是"达到条件"。 这是这套架构跟死程序最根本的区别。

完整逻辑链 v1.0 · 2026-09-19 · 基于两台真机实测